8-3 选择排序

1.简单选择排序

基本思想：每一趟排序在无序序列中找到最小的元素与无序序列的第一个元素交换，直到整体有序

代码表示：

```
void SelectSort(SqList &L){
    //对顺序表L进行简单选择排序
    for(i=1;i<L.length;++i){
    //在L[i..L.length] 中选择关键字最小的记录
        k=i;
        for(j=i+1;j<=L.length;++j){
            if(L[j]<L[k]) k=j;}
        if(k!=i){
            t=L[i];
            L[i]=L[k];
            L[k]=t;}//交换r[i]与r[k]
        }//for
}
```

2.堆排序

大根堆

基本思想：将数组视为一棵完全二叉树，并且每个子树的根节点都大于自己的孩子

堆排序流程：
1. 建堆
2. 每次输出堆顶元素，然后将堆底元素送入堆顶，调整堆来继续保持大根堆性质，再输出堆顶元素，直到整体有序

大根堆建堆步骤：
1. 从最后一个分支节点所在子树开始，根节点与较大孩子互换
2. 逐个从后往前让所有子树满足大根堆定义

代码表示：

```
void CreatHeap(SqList &L){
    // 把无序序列 L[1..n] 建成大根堆
    n = L.length;
    for (i = n / 2; i > 0; --i){
     // 反复调用HeapAdjust
        HeapAdjust(L, i, n);
         }
    }
```

```
void HeapSort(SqList &L){
    //对顺序表 L 进行堆排序
    CreatHeap(L);
    //把无序序列 L[1..L.length] 建成大根堆
    for (i = L.length; i > 1; i--){
        x = L[1];
        //堆顶堆底记录互换
        L[1] = L[i];
        L[i] = x;
        HeapAdjust(L, 1, i - 1);
        //将 L[1..i-1] 重新调整为大根堆
    }//for
}
```

```
void HeapAdjust(SqList &L, int s, int m){
    // 假设L[s+1..m]已经是堆
    //将L[s..m]调整为以L[s]为根的大根堆
    rc = L[s];
    for (j = 2 * s; j <= m; j *= 2){
    //沿key较大的孩子节点向下筛选
        if (j < m && L[j] < L[j + 1]) ++j;
        // j为key较大的记录的下标
        if (rc >= L[j]) break;
            // rc应插入在位置s上
        L[s] = L[j];
        s = j;
    }   // for
    L[s] = rc; // 插入
}
```

小根堆

基本思想：
- 将数组视为一棵完全二叉树
- 并且每个子树的根节点都小于自己的孩子

堆的插入

1. 新元素插到堆尾
2. 向上调整，恢复堆的性质

堆的删除

1. 删除堆顶元素
2. 将最后一个元素换到堆顶
3. 向下调整堆

by joshuji 由 幕布 发布